

Access Control Options

AI Photobooth — 4 ways to restrict who can use the site

Prepared 9 June 2026 · ai-photobooth-zeta.vercel.app

This document compares four practical approaches to limiting access to the AI Photobooth web app to specific people, networks, or devices. Each section explains how the approach works, what it costs, how disruptive it is for end users, and where it shines (or breaks). A side-by-side comparison table is included on the final page so the right choice can be made at a glance.

At a glance

- Option 1 — Password gate: simplest. A shared password, typed once per device and remembered.
- Option 2 — IP allowlist: zero-friction, but only works if all devices stay on a fixed network.
- Option 3 — Device registration: the most controlled option; each device is approved individually.
- Option 4 — Vercel Deployment Protection: zero code, requires a paid Vercel plan.

Option 1 — Password gate

Effort: ~30 minutes · No subscription cost · Most flexible

How it works

When any visitor first opens the site, a small lock screen asks for a password. Once typed correctly the server sends back a signed, HTTP-only cookie that lives on that device for a long time (e.g. 90 days). On every later visit the cookie is checked automatically — the user never sees the lock screen again on that same browser, even after restarting the device.

Strengths

- Works on any device, anywhere in the world.
- No external service or subscription required.
- You can rotate the password at any time; every device is forced to re-enter it.
- Easy to add a separate password for admins (the `/admin.html` page already does this).

Trade-offs

- Anyone who learns the password can share it (a screenshot, an email forward).
- Mitigated by short rotation cycles and by keeping the audience small.
- No per-device revocation — if one person leaks the password, rotation revokes everyone.

Best for

Internal team members, trusted client demos, or a controlled kiosk setup where you can physically visit each device once to type the password in. This is the right choice when the audience is small and trusted.

Setup outline

- Add a `/api/login` endpoint that takes a password, validates against a `SITE_PASSWORD` env var, and sets a signed cookie.
- Add Vercel middleware (or a check inside each function) that 401s any request without the cookie.
- Add a clean lock-screen HTML page.
- Rotate by changing the env var and redeploying — every device falls back to the lock screen.

Option 2 — IP allowlist

Effort: ~20 minutes · No subscription cost · Zero friction (but inflexible)

How it works

Vercel middleware checks the visitor's public IP address on every request. If the IP appears in a short list of approved addresses, the request goes through. Anything else gets blocked with a 403 page. There is no password and no login — the network itself is the credential.

Strengths

- Completely invisible to authorized users — no login, no popup, no clicks.
- Effectively device-tied as long as those devices stay on the approved network.
- Very hard to abuse: an attacker would need to be physically on the same network to even try.

Trade-offs

- The moment a device leaves the network (kiosk goes home, user switches to mobile data), it is locked out.
- Home or public IPs change frequently; only practical for fixed business networks with static IPs.
- Cannot grant access to remote users without a VPN.
- Mobile networks share a small pool of egress IPs across many users — using a phone IP would inadvertently allow many other people.

Best for

A photobooth deployed permanently at one physical venue (a museum lobby, an exhibition stand, an office reception) with one stable public IP. Combine with Option 1 if a few off-network people also need to log in remotely.

Setup outline

- Confirm the venue has a static public IP (your IT / ISP).
- Add a `middleware.js` file in the project root that reads the IP from the `x-forwarded-for` header and matches it against an `ALLOWED_IPS` env var.
- Optionally render a friendly "this site is restricted to specific networks" page instead of a bare 403.

Option 3 — Device registration (allowlist)

Effort: ~1–2 hours · No subscription cost · Most controlled

How it works

Each authorized device is added to a list managed by you. The first time a new device opens the site it generates a unique random identifier locally and sees a "request access" page. You receive that request, approve it from a small admin panel, and from then on the device's identifier is matched against a Firestore allowlist on every request. You can revoke any single device at any time, without affecting anyone else.

Strengths

- True per-device control — revoke one device without disturbing anyone else.
- Travels with the device — works the same on WiFi, mobile data, or while travelling.
- Built-in audit trail: you see exactly which devices have been approved, when, and last seen.
- Resistant to credential sharing — there is no password to leak.

Trade-offs

- Most code to write of the four options (request page, approve flow, allowlist storage).
- A small admin UI is required for approving requests; we already have `/admin.html` and can extend it.
- If a user clears browser storage on an authorized device, the device must re-request access.

Best for

Distributing the photobooth to specific people on their own devices (a roving brand activation with staff phones, a sales team's demo tablets, a partner programme with selected reviewers). The higher effort pays off as soon as the audience grows beyond what a shared password can control.

Setup outline

- Extend Firestore with a "devices" collection: id, name, status, approvedAt, lastSeenAt.
- Client generates a random device ID on first visit, stores it locally, and sends it in a header on every request.
- Middleware allows requests whose device ID is in the "approved" status; everything else routes to a "request access" page.
- Admin panel lists pending and approved devices, with one-click Approve and Revoke actions.

Option 4 — Vercel Deployment Protection

Effort: ~5 minutes (config only) · ~\$20 / month for Vercel Pro · Zero code

How it works

Vercel offers built-in protection at the platform level. With one toggle in the Vercel dashboard, the entire deployment becomes accessible only to people who are signed into a Vercel account that has access to the project (or, on higher tiers, to people who sign in through an external SSO provider you configure). Browsers without a valid Vercel session get redirected to a Vercel login page automatically.

Strengths

- No application code to maintain — it is enforced by the platform itself.
- Strong security: Vercel handles cookies, tokens, and account access.
- Centralized: revoking a team member in Vercel instantly revokes their site access.
- Optional SSO support on higher tiers — users sign in with Google / Microsoft / Okta.

Trade-offs

- Requires a Vercel Pro plan (around USD 20 per user per month at the time of writing).
- Every user needs a Vercel account (or your SSO identity) to access the site.
- Adds a login screen on every new browser session — slightly higher friction than a one-time password.
- Overkill for purely public-facing kiosk traffic, where users will never log in to anything.

Best for

A small internal team that already uses Vercel professionally, where the cost is negligible and the convenience of zero code is worth more than the per-user fee. Not appropriate for end-user kiosks.

Setup outline

- Upgrade the Vercel project to a Pro plan.
- In the Vercel dashboard: Project !' Settings !' Deployment Protection !' enable "Vercel Authentication".
- Invite the people who should have access as team members.
- Optionally configure SSO (SAML / OIDC) on Enterprise tier.

Side-by-side comparison

	Password gate	IP allowlist	Device reg.	Vercel Protection
Effort to set up	~30 min	~20 min	~1–2 hr	~5 min config
Monthly cost	Free	Free	Free	\$20+ / mo
User friction	Low (once)	None	Low (once)	Medium
Travel friendly	Yes	No	Yes	Yes
Per-device revocation	No	No	Yes	Per user
Code complexity	Small	Tiny	Moderate	Zero
Best audience	Trusted team	Fixed venue	Managed fleet	Internal team

Recommendation

For a kiosk at a single fixed venue, Option 2 (IP allowlist) gives the smoothest experience and requires almost no code. For a small trusted team that needs remote access, Option 1 (Password gate) is the right starting point — it can be implemented in an afternoon and rotated easily when needed. If you anticipate handing out access to many specific devices that should be revocable individually, Option 3 (Device registration) is the

more durable investment. Option 4 is only worth considering if a paid Vercel plan is already in place.

Next steps

On approval of one of the options above we can begin implementation. All four are non-destructive and reversible — we can move between them later if requirements change.

